

Django Reference Guide

Hospital Management System Edition

ORM Queries • Forms • Views & Business Logic • Templates • Essential Patterns

Sample models used throughout: Patient, Doctor, Appointment, Medicine, Sale, Bill

0. Sample Models (used in all examples)

Every example in this guide is based on this small hospital data model. Keep it in mind while reading.

```
# models.py
from django.db import models
from django.contrib.auth.models import User

class Doctor(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    speciality = models.CharField(max_length=100) # e.g. "Cardiology"
    fee = models.DecimalField(max_digits=8, decimal_places=2)
    is_active = models.BooleanField(default=True)

class Patient(models.Model):
    name = models.CharField(max_length=100)
    nic = models.CharField(max_length=15, unique=True)
    phone = models.CharField(max_length=12)
    age = models.PositiveIntegerField()
    gender = models.CharField(max_length=10,
                              choices=[("M", "Male"), ("F", "Female")])
    created_at = models.DateTimeField(auto_now_add=True)

class Appointment(models.Model):
    STATUS = [("PENDING", "Pending"), ("DONE", "Done"), ("CANCELLED", "Cancelled")]
    patient = models.ForeignKey(Patient, on_delete=models.CASCADE,
                                related_name="appointments")
    doctor = models.ForeignKey(Doctor, on_delete=models.CASCADE)
    date = models.DateField()
    time = models.TimeField()
    status = models.CharField(max_length=10, choices=STATUS, default="PENDING")

class Medicine(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(max_digits=8, decimal_places=2)
    stock = models.PositiveIntegerField(default=0) # units available
    expiry_date = models.DateField()

class Sale(models.Model):
    medicine = models.ForeignKey(Medicine, on_delete=models.PROTECT)
    quantity = models.PositiveIntegerField()
    total = models.DecimalField(max_digits=10, decimal_places=2)
    sold_at = models.DateTimeField(auto_now_add=True)
```

1. Django ORM — Common Query Filters

The ORM (Object Relational Mapper) lets you query the database with Python instead of SQL. All queries start from **Model.objects** (the manager). Most methods return a **QuerySet**, which is lazy — the database is only hit when you actually use the results.

1.1 .all() — get every row

Returns all records of the table as a QuerySet.

```
patients = Patient.objects.all()           # SELECT * FROM patient
doctors = Doctor.objects.all()[:10]       # first 10 only (LIMIT 10)
```

1.2 .get() — exactly one record

Use when you expect a single row (e.g. by primary key). Raises **DoesNotExist** if not found and **MultipleObjectsReturned** if more than one.

```
patient = Patient.objects.get(id=5)
patient = Patient.objects.get(nic="991234567V")

# Safe version for views (returns 404 page instead of crashing):
from django.shortcuts import get_object_or_404
patient = get_object_or_404(Patient, id=5)
```

1.3 .filter() — WHERE conditions

Returns a QuerySet of all matching rows (can be empty). Conditions combine with AND.

```
# All female patients
Patient.objects.filter(gender="F")

# Cardiology doctors who are active (AND)
Doctor.objects.filter(speciality="Cardiology", is_active=True)

# Today's pending appointments
from datetime import date
Appointment.objects.filter(date=date.today(), status="PENDING")
```

1.4 .exclude() — the opposite of filter

Returns rows that do NOT match.

```
# All appointments except cancelled ones
Appointment.objects.exclude(status="CANCELLED")

# Medicines that are not out of stock
Medicine.objects.exclude(stock=0)
```

1.5 Field lookups — __ (double underscore) magic

Lookups go after the field name and let you do more than exact match. These are the ones you will use daily:

```
Patient.objects.filter(name__icontains="silva") # LIKE '%silva%' (case-insensitive)
Patient.objects.filter(name__startswith="Ka") # name starts with "Ka"
Patient.objects.filter(age__gt=60) # age > 60
Patient.objects.filter(age__gte=60) # age >= 60
Patient.objects.filter(age__lt=18) # age < 18 (children ward)
Patient.objects.filter(age__lte=18) # age <= 18
Patient.objects.filter(age__range=(30, 40)) # BETWEEN 30 AND 40
Patient.objects.filter(id__in=[1, 4, 9]) # id IN (1,4,9)
Medicine.objects.filter(expiry_date__year=2026) # expiry in year 2026
Appointment.objects.filter(date__month=7) # July appointments
Patient.objects.filter(phone__isnull=True) # phone IS NULL
Patient.objects.filter(nic__iexact="991234567v") # exact, ignore case
```

1.6 Query across relationships (ForeignKey)

Use the related field name + __ to filter through a relationship — the ORM writes the JOIN for you.

```
# Appointments of doctors in Cardiology
Appointment.objects.filter(doctor__speciality="Cardiology")

# Appointments where patient name contains "perera"
Appointment.objects.filter(patient__name__icontains="perera")

# Reverse direction: patients who have at least one DONE appointment
Patient.objects.filter(appointments__status="DONE").distinct()
```

1.7 .order_by(), .first(), .last(), .exists(), .count()

```
Patient.objects.order_by("name") # A -> Z
Patient.objects.order_by("-created_at") # newest first (minus = descending)

Appointment.objects.filter(status="PENDING").first() # one object or None
Appointment.objects.order_by("date").last()

Medicine.objects.filter(stock=0).exists() # True/False - very fast check
Patient.objects.count() # total number of patients
```

1.8 .values() and .values_list() — get dicts/tuples instead of objects

Useful for dropdowns, charts and JSON responses.

```
Patient.objects.values("id", "name")
# <QuerySet [{'id': 1, 'name': 'Kasun'}, ...]>

Doctor.objects.values_list("speciality", flat=True).distinct()
# <QuerySet ['Cardiology', 'ENT', ...]>
```

1.9 Q objects — OR / complex conditions

By default `filter()` joins conditions with AND. Use Q for OR and NOT.

```
from django.db.models import Q

# Patients named "silva" OR older than 65
Patient.objects.filter(Q(name__icontains="silva") | Q(age__gt=65))

# Search box: match name OR NIC OR phone
q = request.GET.get("q", "")
Patient.objects.filter(
    Q(name__icontains=q) | Q(nic__icontains=q) | Q(phone__icontains=q)
)

# NOT cancelled
Appointment.objects.filter(~Q(status="CANCELLED"))
```

1.10 Aggregation — count, sum, average

`aggregate()` returns one summary dict; `annotate()` adds a computed column to each row.

```
from django.db.models import Count, Sum, Avg, Max, Min

Sale.objects.aggregate(total_income=Sum("total"))
# {'total_income': Decimal('154300.00')}

Patient.objects.aggregate(avg_age=Avg("age"), oldest=Max("age"))

# Number of appointments per doctor (adds .num_appts to each doctor)
Doctor.objects.annotate(num_appts=Count("appointment")) \
    .order_by("-num_appts")
```

1.11 select_related / prefetch_related — fix slow pages

Without these, looping over appointments and printing `a.patient.name` fires one extra query per row (the N+1 problem). `select_related` does a JOIN for ForeignKey/OneToOne; `prefetch_related` is for reverse FK and ManyToMany.

```
# 1 query instead of 200:
appts = Appointment.objects.select_related("patient", "doctor") \
    .filter(date=date.today())

# Reverse relation: each patient with their appointments preloaded
patients = Patient.objects.prefetch_related("appointments")
```

1.12 Create / Update / Delete quick reference

```
# CREATE
p = Patient.objects.create(name="Nimal", nic="88123V", phone="0771234567",
                           age=38, gender="M")

# get existing OR create if missing
med, created = Medicine.objects.get_or_create(
    name="Paracetamol 500mg",
    defaults={"price": 2.50, "stock": 100, "expiry_date": "2027-01-01"},
)

# UPDATE one object
p.phone = "0719999999"
p.save(update_fields=["phone"])

# BULK update (one SQL statement, no loop needed)
Appointment.objects.filter(date__lt=date.today(), status="PENDING") \
    .update(status="CANCELLED")

# DELETE
Appointment.objects.filter(status="CANCELLED", date__year=2024).delete()
```

2. forms.py — Common Patterns

Django forms handle rendering inputs, validating data, and saving models. **ModelForm** builds a form automatically from a model — use it 90% of the time.

2.1 Basic ModelForm with Bootstrap widgets

fields controls what appears; widgets let you add CSS classes and input types.

```
# forms.py
from django import forms
from .models import Patient, Appointment, Medicine

class PatientForm(forms.ModelForm):
    class Meta:
        model = Patient
        fields = ["name", "nic", "phone", "age", "gender"]
        widgets = {
            "name": forms.TextInput(attrs={"class": "form-control",
                                           "placeholder": "Full name"}),
            "nic": forms.TextInput(attrs={"class": "form-control"}),
            "phone": forms.TextInput(attrs={"class": "form-control"}),
            "age": forms.NumberInput(attrs={"class": "form-control",
                                           "min": 0, "max": 120}),
            "gender": forms.Select(attrs={"class": "form-select"}),
        }
        labels = {"nic": "NIC Number"}
```

2.2 Date / time pickers

Using `type='date'` and `type='time'` gives you native browser pickers with zero JavaScript.

```
class AppointmentForm(forms.ModelForm):
    class Meta:
        model = Appointment
        fields = ["patient", "doctor", "date", "time"]
        widgets = {
            "date": forms.DateInput(attrs={"type": "date",
                                           "class": "form-control"}),
            "time": forms.TimeInput(attrs={"type": "time",
                                           "class": "form-control"}),
        }
```

2.3 Field-level validation — `clean_()`

Runs automatically for that one field. Raise `ValidationError` to show a message under the input.

```
class PatientForm(forms.ModelForm):
    # ... Meta as above ...

    def clean_phone(self):
        phone = self.cleaned_data["phone"]
        if not phone.startswith("07") or len(phone) != 10:
            raise forms.ValidationError("Enter a valid mobile (07XXXXXXXX).")
        return phone

    def clean_age(self):
        age = self.cleaned_data["age"]
        if age > 120:
            raise forms.ValidationError("Age seems invalid.")
        return age
```

2.4 Form-level validation — `clean()` (compare multiple fields)

Use when the rule involves more than one field, e.g. no double-booking a doctor.

```
class AppointmentForm(forms.ModelForm):
    # ... Meta ...

    def clean(self):
        cleaned = super().clean()
        doctor, d, t = cleaned.get("doctor"), cleaned.get("date"), cleaned.get("time")
        if doctor and d and t:
            clash = Appointment.objects.filter(
                doctor=doctor, date=d, time=t
            ).exclude(pk=self.instance.pk) # ignore self when editing
            if clash.exists():
                raise forms.ValidationError(
                    "This doctor already has an appointment at that time."
                )
        return cleaned
```

2.5 Limiting dropdown choices (`queryset`)

Only show active doctors in the appointment form.

```
class AppointmentForm(forms.ModelForm):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.fields["doctor"].queryset = Doctor.objects.filter(is_active=True)
        self.fields["doctor"].empty_label = "-- Select doctor --"
```

2.6 Extra (non-model) fields and plain forms.Form

For things you don't store directly, like a search box or a 'quantity to sell' input.

```
class SellMedicineForm(forms.Form):
    medicine = forms.ModelChoiceField(
        queryset=Medicine.objects.filter(stock__gt=0),
        widget=forms.Select(attrs={"class": "form-select"}))
    quantity = forms.IntegerField(
        min_value=1,
        widget=forms.NumberInput(attrs={"class": "form-control"}))

    def clean(self):
        cleaned = super().clean()
        med, qty = cleaned.get("medicine"), cleaned.get("quantity")
        if med and qty and qty > med.stock:
            raise forms.ValidationError(
                f"Only {med.stock} units of {med.name} left in stock.")
        return cleaned
```

2.7 File / image upload field

Remember enctype='multipart/form-data' in the template and request.FILES in the view.

```
class ReportUploadForm(forms.ModelForm):
    class Meta:
        model = LabReport
        fields = ["patient", "report_file"]
        widgets = {"report_file": forms.ClearableFileInput(
            attrs={"class": "form-control"})}

# view: form = ReportUploadForm(request.POST, request.FILES)
```

3. views.py — Filters, Business Logic & Stock Control

Views connect URLs, forms, the ORM and templates. Below are the patterns used constantly in a hospital system: list + search, create/edit with forms, and safe stock decrement when a sale happens.

3.1 List view with search + filter (GET parameters)

Reads ?q=... and ?speciality=... from the URL and filters the QuerySet step by step.

```
# views.py
from django.shortcuts import render, redirect, get_object_or_404
from django.db.models import Q
from .models import Patient, Doctor, Appointment, Medicine, Sale

def patient_list(request):
    patients = Patient.objects.all().order_by("-created_at")

    q = request.GET.get("q", "")
    if q:
        patients = patients.filter(
            Q(name__icontains=q) | Q(nic__icontains=q) | Q(phone__icontains=q))

    gender = request.GET.get("gender", "")
    if gender in ("M", "F"):
        patients = patients.filter(gender=gender)

    return render(request, "patients/list.html",
                  {"patients": patients, "q": q, "gender": gender})
```

3.2 Pagination

```
from django.core.paginator import Paginator

def patient_list(request):
    qs = Patient.objects.order_by("-created_at")
    paginator = Paginator(qs, 20) # 20 per page
    page = paginator.get_page(request.GET.get("page"))
    return render(request, "patients/list.html", {"page": page})

# template:
# {% for p in page %} ... {% endfor %}
# {% if page.has_next %}<a href="?page={{ page.next_page_number }}">Next</a>{% endif %}
```

3.3 Create + edit with the same pattern

The classic GET-shows-form / POST-saves pattern, with a success message.

```
from django.contrib import messages

def patient_create(request):
    form = PatientForm(request.POST or None)
    if request.method == "POST" and form.is_valid():
        form.save()
        messages.success(request, "Patient registered successfully.")
        return redirect("patient_list")
    return render(request, "patients/form.html", {"form": form})

def patient_edit(request, pk):
    patient = get_object_or_404(Patient, pk=pk)
    form = PatientForm(request.POST or None, instance=patient)
    if request.method == "POST" and form.is_valid():
        form.save()
        messages.success(request, "Patient updated.")
        return redirect("patient_list")
    return render(request, "patients/form.html", {"form": form})
```

3.4 ★ Decrease stock when someone buys (the safe way)

This is the pattern you asked about. Two key tools: **F()** makes the database do the subtraction (stock = stock - qty), so two cashiers selling at the same moment can't overwrite each other. **transaction.atomic** makes the Sale record and the stock change succeed or fail together. **select_for_update()** locks the row until the transaction ends.

```
from django.db import transaction
from django.db.models import F

def sell_medicine(request):
    form = SellMedicineForm(request.POST or None)
    if request.method == "POST" and form.is_valid():
        med = form.cleaned_data["medicine"]
        qty = form.cleaned_data["quantity"]

        with transaction.atomic():
            # Lock this medicine row so nobody else changes it mid-sale
            med = Medicine.objects.select_for_update().get(pk=med.pk)

            if med.stock < qty:
                messages.error(request, f"Only {med.stock} units left.")
                return redirect("sell_medicine")

            # 1) record the sale
            Sale.objects.create(medicine=med, quantity=qty,
                               total=med.price * qty)

            # 2) decrease stock IN THE DATABASE (atomic, race-safe)
            Medicine.objects.filter(pk=med.pk).update(stock=F("stock") - qty)

        messages.success(request, f"Sold {qty} x {med.name}.")
        return redirect("sell_medicine")

    return render(request, "pharmacy/sell.html", {"form": form})
```

Why not **med.stock -= qty; med.save()**? Because between reading and saving, another request may also sell — both read stock=10, both save 9, and one sale is lost. **F('stock') - qty** runs as a single UPDATE in SQL, so it is

always correct.

3.5 Increase stock (restock / purchase from supplier)

Same idea in reverse.

```
def restock(request, pk):
    if request.method == "POST":
        qty = int(request.POST.get("quantity", 0))
        if qty > 0:
            Medicine.objects.filter(pk=pk).update(stock=F("stock") + qty)
            messages.success(request, "Stock updated.")
        return redirect("medicine_list")
```

3.6 Dashboard view — counts and sums in one place

```
from datetime import date
from django.db.models import Sum, Count

def dashboard(request):
    today = date.today()
    ctx = {
        "patient_count": Patient.objects.count(),
        "todays_appts": Appointment.objects.filter(date=today).count(),
        "pending": Appointment.objects.filter(status="PENDING").count(),
        "low_stock": Medicine.objects.filter(stock__lt=10),
        "expired": Medicine.objects.filter(expiry_date__lt=today),
        "today_income": Sale.objects.filter(sold_at__date=today)
            .aggregate(t=Sum("total"))["t"] or 0,
    }
    return render(request, "dashboard.html", ctx)
```

3.7 Restricting views — login and roles

```
from django.contrib.auth.decorators import login_required, user_passes_test

@login_required
def dashboard(request):
    ...

def is_doctor(user):
    return hasattr(user, "doctor")

@login_required
@user_passes_test(is_doctor)
def my_appointments(request):
    appts = Appointment.objects.filter(doctor=request.user.doctor,
                                       status="PENDING")
    return render(request, "doctor/appointments.html", {"appts": appts})
```

3.8 Change status (e.g. mark appointment done) + JSON response

```
from django.http import JsonResponse

def mark_done(request, pk):
    appt = get_object_or_404(Appointment, pk=pk)
    appt.status = "DONE"
    appt.save(update_fields=["status"])
    return redirect("appointment_list")

# AJAX version (for fetch() from JavaScript)
def medicine_stock_api(request, pk):
    med = get_object_or_404(Medicine, pk=pk)
    return JsonResponse({"name": med.name, "stock": med.stock,
                        "price": float(med.price)})
```

4. Django Templates — Common Codes

Note: 'Blade' is Laravel's template engine — Django's equivalent is the **Django Template Language (DTL)**. Same idea, different syntax: `{{ variable }}` prints a value and `{% tag %}` is logic.

4.1 Template inheritance (layout + child pages)

Define one base layout, then every page extends it — like `@extends/@yield` in Blade.

```

<!-- templates/base.html -->
<!DOCTYPE html>
<html>
<head>
  <title>{% block title %}Hospital MS{% endblock %}</title>
  {% load static %}
  <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
  {% include "partials/navbar.html" %}
  <div class="container">
    {% block content %}{% endblock %}
  </div>
</body>
</html>

<!-- templates/patients/list.html -->
{% extends "base.html" %}
{% block title %}Patients{% endblock %}
{% block content %}
  <h2>Patients</h2>
  ...
{% endblock %}

```

4.2 Loops, conditions, empty state

```

<table class="table">
  {% for p in patients %}
    <tr>
      <td>{{ forloop.counter }}</td>
      <td>{{ p.name }}</td>
      <td>{{ p.age }}</td>
      <td>
        {% if p.age >= 60 %}
          <span class="badge bg-warning">Senior</span>
        {% elif p.age < 18 %}
          <span class="badge bg-info">Child</span>
        {% else %}
          <span class="badge bg-success">Adult</span>
        {% endif %}
      </td>
    </tr>
  {% empty %}
    <tr><td colspan="4">No patients found.</td></tr>
  {% endfor %}
</table>

```

4.3 Rendering a form + CSRF + errors

csrf_token is required on every POST form or Django will reject it (403).

```
<form method="post" enctype="multipart/form-data">
  {% csrf_token %}

  {{ form.non_field_errors }}

  {% for field in form %}
    <div class="mb-3">
      <label class="form-label">{{ field.label }}</label>
      {{ field }}
      {% if field.errors %}
        <div class="text-danger small">{{ field.errors.0 }}</div>
      {% endif %}
    </div>
  {% endfor %}

  <button type="submit" class="btn btn-primary">Save</button>
</form>
```

4.4 URLs, links and flash messages

```
<a href="{% url 'patient_edit' p.id %}" class="btn btn-sm btn-outline-primary">
  Edit
</a>

{% if messages %}
  {% for m in messages %}
    <div class="alert alert-{{ m.tags }}">{{ m }}</div>
  {% endfor %}
{% endif %}

{% if user.is_authenticated %}
  Hello {{ user.username }} | <a href="{% url 'logout' %}">Logout</a>
{% else %}
  <a href="{% url 'login' %}">Login</a>
{% endif %}
```

4.5 Useful template filters

Filters transform values with a pipe | — like Blade's {{ \$x | upper }} concept.

```
{{ p.name|upper }}           KASUN PERERA
{{ p.name|title }}           Kasun Perera
{{ p.created_at|date:"Y-m-d H:i" }} 2026-07-19 09:30
{{ appointment.date|date:"D, d M Y" }} Sun, 19 Jul 2026
{{ bill.total|floatformat:2 }} 1250.00
{{ note|truncatechars:50 }}    cuts long text...
{{ patients|length }}         number of items
{{ med.stock|default:"0" }}    fallback value
{{ description|linebreaks }}   \n -> <p>/<br>
{{ price|floatformat:2 }} LKR

{# comments look like this and are not rendered #}
```

4.6 Highlight low stock in the table (real example)

```
{% for m in medicines %}
|  |  |
| --- | --- |
|{{ m.name }}
 {{ m.price|floatformat:2 }} | {{ m.stock }}         {% if m.stock == 0 %}         <span class="badge bg-danger">Out of stock</span>         {% elif m.stock < 10 %}         <span class="badge bg-warning">Low</span>         {% endif %} |

{% endfor %}
```

5. Beyond CRUD — Other Things You Should Know

You already know CRUD. These are the next skills that make a Django app feel professional.

5.1 Model methods and properties

Put logic on the model so every view and template can reuse it.

```
class Medicine(models.Model):
    # ... fields ...

    def __str__(self):
        # shown in admin & dropdowns
        return f"{self.name} (Rs.{self.price})"

    @property
    def is_low_stock(self):
        return self.stock < 10

    @property
    def is_expired(self):
        from datetime import date
        return self.expiry_date < date.today()

# template: {% if m.is_low_stock %} ... {% endif %}
```

5.2 Meta options — default ordering & unique together

```
class Appointment(models.Model):
    # ... fields ...
    class Meta:
        ordering = ["date", "time"] # default order everywhere
        unique_together = [("doctor", "date", "time")] # DB-level protection
```

5.3 urls.py patterns

```
# app/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path("", views.dashboard, name="dashboard"),
    path("patients/", views.patient_list, name="patient_list"),
    path("patients/add/", views.patient_create, name="patient_create"),
    path("patients/<int:pk>/edit/", views.patient_edit, name="patient_edit"),
    path("api/medicine/<int:pk>/", views.medicine_stock_api,
        name="medicine_api"),
]

# project/urls.py
from django.urls import path, include
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("hospital.urls")),
    path("accounts/", include("django.contrib.auth.urls")), # login/logout
]
```

5.4 Django admin — free back office

Register models and you get a full data management UI instantly.

```
# admin.py
from django.contrib import admin
from .models import Patient, Doctor, Appointment, Medicine

@admin.register(Medicine)
class MedicineAdmin(admin.ModelAdmin):
    list_display = ("name", "price", "stock", "expiry_date")
    list_filter = ("expiry_date",)
    search_fields = ("name",)
    list_editable = ("price", "stock")

@admin.register(Appointment)
class AppointmentAdmin(admin.ModelAdmin):
    list_display = ("patient", "doctor", "date", "time", "status")
    list_filter = ("status", "doctor")
    date_hierarchy = "date"
```

5.5 Signals — run code automatically on events

Example: create a Doctor profile automatically when a staff User is created, or log every sale.

```
# signals.py
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Sale

@receiver(post_save, sender=Sale)
def after_sale(sender, instance, created, **kwargs):
    if created and instance.medicine.stock < 10:
        # send alert email, write a log, notify pharmacist, etc.
        print(f"LOW STOCK: {instance.medicine.name}")

# apps.py -> ready(): from . import signals
```

5.6 Custom template context everywhere (context processor)

Example: show pending-appointment count in the navbar on every page.

```
# hospital/context_processors.py
from .models import Appointment

def navbar_counts(request):
    return {"pending_count":
            Appointment.objects.filter(status="PENDING").count()}

# settings.py -> TEMPLATES.OPTIONS.context_processors +=
# ["hospital.context_processors.navbar_counts"]
# navbar: Pending ({{ pending_count }})
```

5.7 settings.py essentials for deployment

```
import os
DEBUG = os.environ.get("DEBUG", "0") == "1"      # never True in production
ALLOWED_HOSTS = ["yourhospital.com", "localhost"]

STATIC_URL = "/static/"
STATIC_ROOT = BASE_DIR / "staticfiles"           # collectstatic target
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"                  # uploaded files

TIME_ZONE = "Asia/Colombo"
USE_TZ = True
```

5.8 Commands you will use daily

```
python manage.py makemigrations      # detect model changes
python manage.py migrate              # apply to database
python manage.py createsuperuser     # admin login
python manage.py runserver           # dev server
python manage.py shell               # test ORM queries interactively
python manage.py collectstatic       # gather static files for production
python manage.py dumpdata > backup.json
python manage.py loaddata backup.json
```

5.9 Environment variables & secrets (do this!)

Never commit SECRET_KEY or DB passwords. Use django-environ or os.environ.

```
# pip install django-environ
import environ
env = environ.Env()
environ.Env.read_env()           # reads .env file

SECRET_KEY = env("SECRET_KEY")
DATABASES = {"default": env.db("DATABASE_URL")}
# .env -> SECRET_KEY=xxxx
#       DATABASE_URL=postgres://user:pass@localhost/hospital
```

5.10 Quick checklist for your hospital project

```
[ ] F() + transaction.atomic for every stock/quantity change
[ ] select_related on all appointment list pages
[ ] unique_together to block double bookings at DB level
[ ] @login_required on every internal view
[ ] messages framework for user feedback after every POST
[ ] Paginator on lists (patients, sales, appointments)
[ ] Admin registered for every model (free back office)
[ ] Low-stock + expiry alerts on dashboard
[ ] Daily dumpdata backup (cron)
```